RELATÓRIO DO PROJETO AVALIATIVO

Sistema de Gerenciamento de Vendas em Python

Discente: Ian dos Santos | Disciplina: Lógica de Programação | 2026

1. Objetivo

O objetivo do projeto foi desenvolver um sistema de gerenciamento de vendas para uma pequena loja, utilizando Python 3 e execução pelo terminal. A aplicação permite cadastrar e consultar produtos, registrar compras, aplicar descontos, controlar o estoque e emitir relatórios. O trabalho também demonstra o uso de estruturas condicionais, repetições, funções, listas, dicionários e recursividade.

2. Visão geral da solução

O programa apresenta um menu com seis funcionalidades e a opção de encerrar. Os dados são armazenados durante a execução em duas listas principais: produtos e vendas. Cada produto possui nome, preço e quantidade; cada venda guarda os itens adquiridos, o valor bruto, o desconto e o total pago. O carrinho temporário permite selecionar mais de um produto antes da confirmação.

O fluxo geral do sistema é: exibição do menu, escolha da função, execução e validação da operação e retorno ao menu. A opção 0 encerra o programa.

3. Principais funções implementadas

Função	Responsabilidade
cadastrar_produto()	Recebe nome, preço e quantidade, valida as entradas e salva o cadastro.
listar_produtos()	Percorre a lista e mostra nome, preço e quantidade em estoque.
realizar_venda()	Organiza o carrinho, verifica o estoque, solicita confirmação e registra a venda.
calcular_desconto(valor)	Recebe o valor da compra e retorna o desconto conforme a faixa.
calcular_faturamento(lista_vendas)	Soma os valores finais das vendas e retorna o faturamento.
gerar_relatorio()	Mostra o número de vendas, o faturamento total e o histórico de compras.
contagem_regressiva(n)	Exibe os números de n a 0 por recursividade, com condição de parada.

As funções calcular_desconto(valor) e calcular_faturamento(lista_vendas) recebem parâmetros e retornam valores, conforme o requisito de modularização.

4. Estruturas de controle e validação

O menu permanece ativo por meio de um while True e só é encerrado quando o usuário escolhe 0. A seleção das opções utiliza if, elif e else; opções inexistentes produzem uma mensagem de erro. O comando for percorre produtos, itens do carrinho e registros de vendas. Blocos try/except tratam entradas numéricas inválidas, evitando que o sistema seja encerrado por esse motivo.

No cadastro, o nome não pode estar vazio, o preço deve ser maior ou igual a zero e a quantidade precisa ser um número inteiro não negativo. Na venda, o programa valida o produto escolhido, a quantidade solicitada e o estoque disponível, levando em conta as unidades já reservadas no carrinho. O cancelamento preserva o estoque; após a confirmação, a venda é registrada e as quantidades são atualizadas.

4.1. Política de descontos

A função calcular_desconto(valor) aplica a regra abaixo ao valor bruto da compra:

Valor bruto	Desconto	Exemplo: total final
Abaixo de R$ 100,00	0%	R$ 80,00 → R$ 80,00
De R$ 100,00 a R$ 299,99	5%	R$ 200,00 → R$ 190,00
A partir de R$ 300,00	10%	R$ 400,00 → R$ 360,00

5. Recursividade

Recursividade é o processo em que uma função chama a si mesma. No projeto, contagem_regressiva(n) imprime o número recebido e, quando ele ainda é maior que zero, chama a própria função com n - 1. O caso-base é n == 0: nesse ponto, a função retorna e encerra as chamadas, evitando uma repetição infinita.

```
def contagem_regressiva(n):
    print(n)
    if n == 0:  # caso-base
        return
    contagem_regressiva(n - 1)
```

Por exemplo, para n = 5, a saída é 5, 4, 3, 2, 1, 0. Antes da chamada, a opção 6 do menu verifica se o valor digitado é um número inteiro não negativo. A função recursiva não utiliza for nem while.

5.1. Organização dos dados

As listas e os dicionários permitem organizar e consultar as informações sem banco de dados. Como os registros ficam apenas na memória, os produtos cadastrados e as vendas realizadas não permanecem salvos depois que o programa é encerrado.

6. Evidências de funcionamento

Foram executados testes com valores de compra, validação de estoque, entradas do menu, carrinho com múltiplos itens, relatório e recursividade. Os resultados observados no arquivo sgv.py estão resumidos a seguir:

Cenário testado	Resultado observado	Situação
Compra de R$ 80,00	Desconto de R$ 0,00; total de R$ 80,00.	Conforme
Compra de R$ 200,00	Desconto de R$ 10,00; total de R$ 190,00.	Conforme
Compra de R$ 400,00	Desconto de R$ 40,00; total de R$ 360,00.	Conforme
Quantidade acima do estoque	Pedido de 99 unidades rejeitado.	Conforme
Opção inválida (9)	Mensagem “Opção inválida!”.	Conforme
Contagem com n = 5	Saída: 5, 4, 3, 2, 1, 0.	Conforme
Escolha da opção 0	Mensagem “Sistema encerrado!”.	Conforme
Carrinho com dois produtos	2 × R$ 25,00 + 3 × R$ 8,00 = R$ 74,00; estoques atualizados.	Conforme
Relatório após três compras	3 vendas; faturamento de R$ 630,00.	Conforme

6.1. Trechos da saída registrada

```
===== RELATÓRIO DE VENDAS =====
Número de vendas: 3
Faturamento total: R$ 630.00

Valor bruto: R$ 400.00
Desconto: R$ 40.00
Total a pagar: R$ 360.00
```

Os trechos foram retirados da saída do programa nos testes. O terminal apresenta ponto como separador decimal nos valores monetários.

7. Conclusão

O sistema desenvolvido atende às funcionalidades centrais da atividade: cadastro e listagem de produtos, venda com controle de estoque, aplicação de descontos, relatório de faturamento e contagem regressiva recursiva. Os testes realizados confirmaram os resultados esperados nos cenários verificados. O projeto permitiu aplicar, de forma prática, os conteúdos de Lógica de Programação, com destaque para a validação de dados e a divisão do código em funções.

Base do relatório: código-fonte sgv.py e enunciado do Projeto Avaliativo de Lógica de Programação.